

Vergleich der Performanz

Benchmark-Setup

Um zwischen den in Abschnitt **2.2.1 Methoden** vorgestellten Datenverarbeitungsmethoden abzuwägen, haben wir deren Performanz in einem systematischen Benchmark-Experiment auf dem SAS-Server des Statistischen Bundesamtes und den neuen OnSite R- und Stata-Servern des FDZ gemessen. Weiterführende Benchmarks sowie der vollständig reproduzierbare Code sind auf Opencode verfügbar¹.

Für die Untersuchung wurden simulierte Einzeldaten in verschiedenen Größen (0,1%/1%/10% von den insgesamt 80 Mio. Beobachtungen des BTP) betrachtet. Sie wurden aus öffentlich verfügbaren Informationen generiert, um die wesentlichen Strukturen des BTP für unsere Analyse künstlich nachzubilden. Abgrenzend zu den im FDZ bereitgestellten *Datenstrukturfiles* wurden die Testdaten nicht aus echten Mikrodaten abgeleitet. Der simulierte Datensatz ist ausschließlich für technische Testzwecke der nachfolgenden Anwendungsfälle bestimmt:

1. **Fallzahlen** (pro Statistik und Jahr),
2. **Regression** (Einflussfaktoren auf Verlustvorträge).

Um eine zuverlässige Performanz-Messung zu erhalten, wurde jede Datenverarbeitungsmethode dreifach angewandt. Der Vergleich erfolgt anhand der mittleren Messung der folgenden Zielgrößen:

1. **Arbeitsspeicher** der Auswertung,
2. **Laufzeit** der Auswertung (real verstrichene Zeit),
3. **Festplattenspeicher** der verwendeten Daten.

Die Zielgrößen sind in dieser Reihenfolge zu priorisieren, da ein übermäßiger Arbeitsspeicherverbrauch zu schwerwiegenden Abbrüchen führen kann, während hohe Laufzeiten dennoch zu einem erfolgreichen Ergebnis führen können. Festplattenspeicher ist ausreichend vorhanden, aber unnötige Belegung sollte vermieden werden.

Feine Abstufungen der Zielgrößen sind für unsere Aufgaben nicht gleichermaßen relevant. Eine Methode, die die betrachteten Auswertungen in unter 10 Sekunden mit weniger als 4 GB Arbeitsspeicher durchführen kann, wird bereits als optimal bewertet. Weitere Optimierungen sind in diesem Fall nicht erforderlich. Falls bereits die gewünschte Performanz erreicht wurde, rückt die Reduktion des Entwicklungsaufwand in den Vordergrund. Entsprechend sind Aspekte wie einfache Anwendbarkeit und Nachvollziehbarkeit – selbst für unerfahrene Nutzende – ausschlaggebend.

¹www.opencode.de/URL-To-Be-Announced.

Table 1: Baselines in SAS und Stata (anhand 1% des simuliertes BTP)

Software	Anwendung	Arbeitsspeicher	Laufzeit	Festplattenspeicher
SAS	Fallzahl	65 MB	2,32 Min.	33 GB
SAS	Regression	tba	2,40 Min.	33 GB
Stata	Fallzahl	33 GB	55,2 Sek.	32 GB
	Regression	33 GB	55,8 Sek.	32 GB

Baselines in SAS und Stata

Klassisch werden die betrachteten Auswertungsaufgaben im Statistischen Bundesamt hauptsächlich mit SAS und im FDZ mit Stata gelöst. Um die nachfolgenden Ergebnisse einzuordnen, haben wir die Performanzmessungen für 1% des simulierten BTP-Daten (etwa. 800.000 Beobachtungen) in SAS und Stata durchgeführt.

Die Ergebnisse in Table 1 zeigen, dass der umfangreiche Arbeitsspeicher des FDZ OnSite-Servers nicht ausreicht, um das ganze BTP in Stata einzulesen, sodass zwingend mit Variablenselektion gearbeitet werden muss. SAS zeigt seine Stärke anhand der geringen Beanspruchung des Arbeitsspeichers, benötigt aber mit etwa 2,3 Minuten pro Auswertung die doppelte Laufzeit gegenüber Stata.

Kleiner Vergleich zwischen R und den Baselines

Für 1% der simulierten BTP Daten, reduzieren die R-Datenverarbeitungsmethode alle Zielgrößen gegenüber SAS und Stata erheblich (siehe Table 2 und Table 3). Selbst klassische R-Methoden sind um den Faktor 7 schneller und benötigen nur 33 % des Festplattenspeichers. {polars}, {duckdb} und {arrow} lösen die betrachteten Auswertungen sogar

- in **unter 2 Sekunden** (27-fach schneller),
- mit **unter 300 MB Arbeitsspeicher** – {duckdb} und {polars} sogar mit **unter 5 MB** (10-fach weniger)²³ –
- und nur mit **5 GB Festplattenspeicher** (6-fach weniger gegenüber SAS und Stata).

Unter den klassischen R-Methoden bietet {data.table} die beste Performanz. Dabei ist wesentlich zu beachten, dass die Einlesefunktion **fread** direkt die relevanten Variablen auswählt. Dadurch lassen sich unsere Auswertungen um einen Faktor 3 beschleunigen und vor allem der Arbeitsspeicher um einen Faktor 1.000 reduzieren.

Alle hoch-performanten Methoden sind dermaßen effizient, dass die Datenmenge von 800.000 Beobachtungen nicht mehr ausreicht, um zwischen den besten Methoden zu differenzieren.

²Lediglich in der Arbeitsspeicherbeanspruchung, ist SAS deutlich besser als klassische R-Methoden. Gegenüber hoch-performanten R-Methoden benötigt SAS jedoch den 10-fachen Arbeitsspeicher.

³{arrow} benötigt mit 274 MB deutlich mehr Arbeitsspeicher als {polars} und {duckdb}, aber kommt dennoch mit minimalen Entwicklungsumgebungen aus, bspw. einem standardmäßigen Laptop.

Table 2: Performanz Messungen f<U+00FC>r die Fallzahlberechnung anhand 1% des BTP.
tba: mehr

Package	Dateiformat	Optimierung	Arbeitsspeicher	Laufzeit	Festplatten
{polars}	Parquet	-	3,3 MB	1,0 Sek	
{duckdb}	Parquet	-	4,8 MB	1,5 Sek	
{arrow}	Parquet	Variablenauswahl	273,7 MB	1,7 Sek	
{data.table}	CSV	Variablenauswahl	42,9 MB	2,5 Sek	
{arrow}	Parquet	Datenaufteilung (gleichm<U+00E4><U+00DF>ig)	229,7 MB	2,8 Sek	
{arrow}	Parquet	-	272,8 MB	2,8 Sek	
{arrow}	Parquet	-	230,4 MB	3,8 Sek	
{arrow}	Parquet	Datenaufteilung (Berichtsjahr)	229,9 MB	3,9 Sek	
{data.table}	CSV	-	33,7 GB	7,4 Sek	
{data.table}	CSV	Datenaufteilung (Berichtsjahr)	50,4 GB	13,6 Sek	

Table 3: Performanz Messungen f<U+00FC>r die Fallzahlberechnung anhand 1% des BTP.
tba: mehr

Package	Dateiformat	Optimierung	Arbeitsspeicher	Laufzeit	Festplattenspeicher
{arrow}	Parquet	Datenaufteilung (Berichtsjahr)	529,7 MB	3,8 Sek	4,8 GB
{dplyr}	CSV	Variablenauswahl	154,8 MB	13,2 Sek	13,0 GB

Daher erhöhen wir im nächsten Experiment die Datenmenge auf 8.000.000 Beobachtungen (10% BTP).

Table 4: Performanz Messungen f<U+00FC>r die Fallzahlberechnung anhand 10% des BTP.

Package	Dateiformat	Optimierung	Arbeitsspeicher	Laufzeit	Festplatten
{arrow}	Parquet	Datenaufteilung (gleichm<U+00E4><U+00DF>ig)	229,9 MB	4,7 Sek	
{duckdb}	Parquet	-	4,7 MB	5,1 Sek	
{arrow}	Parquet	Variablenauswahl	702,1 MB	7,5 Sek	
{arrow}	Parquet	Datenaufteilung (Berichtsjahr)	4,7 MB	11,7 Sek	
{arrow}	Parquet	-	230,4 MB	17,0 Sek	
{arrow}	Parquet	-	701,2 MB	20,6 Sek	
{data.table}	CSV	Variablenauswahl	427,5 MB	23,6 Sek	
{polars}	Parquet	-	3,3 MB	1,2 Min	
{data.table}	CSV	-	336,0 GB	2,2 Min	
{data.table}	CSV	Datenaufteilung (Berichtsjahr)	506,4 GB	5,0 Min	

Größere Datenmengen in R

Die Betrachtung von 10% der simulierten BTP-Daten, zeigt, dass {arrow}+{parquet} die geringste Laufzeit für unsere Zwecke benötigt und ebenso die Entwicklungszeit minimiert hat. Somit bietet {arrow} im Gesamtpaket die beste Datenverarbeitungsmethode für die Bewältigung unserer Aufgaben.

- In der relevanten Größenordnung ist {arrow} die schnellste Methode (siehe Table 4 und Table 5)
- {arrow} spart ausreichend Arbeitsspeicher (Verbrauch unter 1 GB) (tba: Fußnote zu duckdb/polars). Der Arbeitsspeicherverbrauch von {arrow} ist in unserer Auswertung nahezu konstant abhängig von der Datenmenge, sodass {arrow} leicht auf noch größere Datenmengen skalieren kann.
- {parquet} spart gegenüber CSV mind. 50% der Festplatte (für das reale BTP sogar 80%+)
- {arrow} zeigte im Experiment die beste Nutzendenerfahrung.
 - Beide Anwendungen lassen sich in unter 20 Z. mit Arrow programmieren (verständlich lesbar) s. Opencode Repo.
 - Leicht und verständlich - adaptiert nativ schöne {tidyverse} Syntax. Dagegen sind {duckdb} und {polars} auf Zusatzpakete angewiesen, die tlw. die Performanz reduzieren können.
 - Flexibel, selektives Einlesen ist zwingend erforderlich
 - hohe Stabilität, fehlende Features können durch leicht in duckdb oder in den Arbeitsspeicher überführt und dennoch ausgeführt werden
- kleiner Nachteile: Erfahrung notwendig, da...
 - erst Datenaufteilung/selektives Einlesen die maximale Performanz erreicht
 - aktuell nicht der volle Funktionsumfang von {tidyverse} in {arrow} abgebildet wird, sodass auf fehlende Features mit {duckdb} etc. reagiert werden muss.

Table 5: Performanz Messungen f<U+00FC>r die Regressionsberechnung anhand 10% des BTP.

Package	Dateiformat	Optimierung	Arbeitsspeicher	Laufzeit	Festplattenspeicher
{arrow}	Parquet	Datenaufteilung (Berichtsjahr)	782,1 MB	5,2 Sek	48,1 GB
{dplyr}	CSV	Variablenauswahl	1,5 GB	1,9 Min	130,2 GB

Erkenntnisse des Experiments

- Auswahl der optimalen hoch-performanten Methode erfordert eine Untersuchung in der eigenen Infrastruktur anhand eigener Daten. Jedoch bieten alle HP-Methoden für die meisten Anwendungsfälle starke Performanzgewinne und sollten eingesetzt werden (nach den vorhandenen Erfahrungen und Strukturen: tidy vs Python vs SQL). Falls die Daten in CSV unter 25% des Arbeitsspeichers sind, sind auch klassische Methoden sehr gut nutzbar (bevorzugt {data.table}).
 - **Package für optimale Laufzeit ist nicht eindeutig.** Je nach Datenmenge hat sich die Top 3 stark unterschieden.
 - * 1e6: arrow > duckdb > DT+SEL > polars
 - * 1e5: polars > duckdb > arrow > DT+SEL
 - * 1e4: polars > data.table > arrow > readr
 - Unerwarteterweise ist {polars} bis 1% BTP am schnellsten aber für das 10% BTP abgeschlagen (unter den schlechtesten Methoden).
- Dagegen ist **das effizienteste betrachtete Datenformat über 1000 Beobachtungen eindeutig Parquet** – sowohl in Speicherbedarf als auch in Lese- und Schreibzeiten.
- HP-Methoden und Optimierungen können erst ab einer Mindestdatenmenge vorteilhaft sein. Insb. Datenaufteilung bietet nur HP Methoden ab einer gewissen Datenmenge Vorteile. Für kleine Mengen überwiegt der Overhead durch das Lesen mehrerer kleiner Dateien. Klassische Methoden werden durch eine Datenaufteilung nur negativ beeinflusst.
- **Variablenauswahl beim Einlesen ist immer vorteilhaft.** High-Performance Methoden können aber bei einer **Datenaufteilung** darauf verzichten.
- (tba: mit obigem zusammenführen) Unter den klassischen Methoden ist {data.table} mit Variablenauswahl am effizientesten und schneller als eine schlechte Anwendung von {arrow}. D.h. ein {data.table} Profi, ist besser als ein schlechter Arrow Programmierer, der aber größeres Potenzial für Verbesserungen hat. Anfängern ist eher {arrow} zu empfehlen, da in {data.table} die Variablenauswahl abhängig von dem verfügbaren Arbeitsspeicher manuell gesteuert werden muss und daher nicht so flexibel ist.
 - 5x langsamer als {arrow}
 - Variablenauswahl ab einer gewissen Datenmenge erforderlich, da ohne 3x größer als die Eingangsdaten, dauert 2.2 min anstatt 23 Sec. -> schwierige Anwendung (Entscheidung relevanter Variablen muss vorab getroffen werden unter Beachtung der Arbeitsspeicher Restriktionen)

Table 6: Benchmarkergebnisse für Fallzahlen (kleine Stichproben des BTP)

Beob.	Package	Dateiformat	Optimierung	Arbeitsspeicher	Laufzeit	Fest
10.000	{polars}	Parquet	-	3,3 MB	0,4 Sek	
10.000	{data.table}	CSV	Variablenauswahl	4,4 MB	0,7 Sek	
10.000	{arrow}	Parquet	Variablenauswahl	230,7 MB	1,1 Sek	
10.000	{readr}	CSV	Variablenauswahl	9,3 MB	1,2 Sek	
10.000	{arrow}	Parquet	-	229,8 MB	1,6 Sek	
10.000	{duckdb}	Parquet	-	4,7 MB	2,9 Sek	
10.000	{data.table}	CSV	-	3,4 GB	3,6 Sek	
10.000	{arrow}	Parquet	Datenaufteilung (Berichtsjahr)	229,9 MB	4,2 Sek	
10.000	{arrow}	Parquet	-	230,4 MB	4,6 Sek	
10.000	{data.table}	CSV	Datenaufteilung (Berichtsjahr)	5,1 GB	4,8 Sek	
10.000	{arrow}	Parquet	Datenaufteilung (gleichm<U+00E4><U+00DF>ig)	229,9 MB	7,1 Sek	
10.000	{readr}	CSV	-	1,7 GB	8,6 Sek	
10.000	{arrow}	CSV	-	27,4 MB	15,7 Sek	
10.000	{data.table}	CSV.GZ	-	4,7 GB	21,2 Sek	
10.000	{base}	CSV	-	9,7 GB	4,9 Min	
1.000	{data.table}	CSV	Variablenauswahl	0,6 MB	0,1 Sek	
1.000	{readr}	CSV	Variablenauswahl	2,7 MB	0,2 Sek	
1.000	{polars}	Parquet	-	3,3 MB	0,4 Sek	
1.000	{data.table}	CSV	-	341,8 MB	0,9 Sek	
1.000	{duckdb}	Parquet	-	4,7 MB	1,1 Sek	
1.000	{arrow}	Parquet	Variablenauswahl	226,2 MB	1,2 Sek	
1.000	{arrow}	Parquet	-	225,3 MB	1,4 Sek	
1.000	{arrow}	CSV	-	3,5 MB	1,5 Sek	
1.000	{data.table}	CSV.GZ	-	491,7 MB	2,7 Sek	
1.000	{data.table}	CSV	Datenaufteilung (Berichtsjahr)	510,9 MB	2,8 Sek	
1.000	{arrow}	Parquet	Datenaufteilung (Berichtsjahr)	229,9 MB	2,9 Sek	
1.000	{arrow}	Parquet	-	230,4 MB	3,7 Sek	
1.000	{arrow}	Parquet	Datenaufteilung (gleichm<U+00E4><U+00DF>ig)	229,7 MB	4,0 Sek	
1.000	{readr}	CSV	-	197,4 MB	4,9 Sek	
1.000	{base}	CSV	-	713,4 MB	23,3 Sek	

– Datenaufteilung verschlechtert DT im Gegensatz zu arrow

- {arrow} verzeiht Fehler leicht(er als DT). Denn schlecht programmiert, ist es immernoch ganz gut). Alleine durch die Datenaufteilung (vorgegeben durch die Datenbereitstellung, vorbereitet durch das FDZ), erreichen wir die Fallzahl-Berechnung in 11.73s, nur 5s über dem absoluten Optimum.
- R-base oder readr sind immer weit abgeschlagen und zu vermeiden.
- Wenn der RAM extrem begrenzt ist, können {polars} oder {duckdb} dennoch eingesetzt werden.

Anhang

Weitere Performancemessungen für kleinere Datenmengen.

Table 7: Benchmarkergebnisse f<U+00FC>r Regression (kleine Stichproben des BTP)

Beob.	Package	Dateiformat	Optimierung	Arbeitsspeicher	Laufzeit	Festplattenspeicher
10.000	{dplyr}	CSV	Variablenauswahl	17,6 MB	1,3 Sek	1,3 GB
10.000	{arrow}	Parquet	Datenaufteilung (Berichtsjahr)	459,3 MB	5,0 Sek	486,9 MB
10.000	{dplyr}	CSV	-	1,8 GB	10,8 Sek	1,3 GB
1.000	{dplyr}	CSV	Variablenauswahl	4,2 MB	0,3 Sek	129,8 MB
1.000	{dplyr}	CSV	-	206,9 MB	6,4 Sek	129,8 MB
1.000	{arrow}	Parquet	Datenaufteilung (Berichtsjahr)	452,5 MB	6,8 Sek	53,5 MB